PYTHON PROGRAMS

10 Problem-Solving Topics with Algorithms, Flowcharts, Pseudocode, Source Code, Sample Outputs, Test Cases and Conclusions

Programs included

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservations Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

1. University Scholarship Allocation

Problem Statement

Evaluate scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and family-income criteria. Rank eligible students and resolve ties using predefined rules.

Objectives

• Check every eligibility rule consistently.
• Rank eligible applicants by aggregate marks, then attendance, then lower family income.
• Display eligibility and ranking clearly.

Assumptions / Rules

• Minimum aggregate = 80%; minimum score in every subject = 70%; attendance = 85% or more; annual family income ≤ ■2,50,000.
• For ties: higher attendance ranks first; if still tied, lower family income ranks first.
• Sample data is built into the program so it runs without extra input.

Algorithm

1. Start.
2. Store applicant details, including subject marks, attendance, and annual family income.
3. Check aggregate, every subject minimum, attendance, and income.
4. Separate eligible applicants from ineligible applicants.
5. Sort eligible applicants by aggregate descending, attendance descending, and income ascending.
6. Display the eligibility result and ranked list.
7. Stop.

Flowchart (vertical sequence)

Start
↓
Set eligibility rules
↓
Read applicant records
↓
Check marks, attendance and income
↓
Separate eligible applicants
↓
Rank eligible applicants
↓
Display results
↓
Stop

Pseudocode

```
BEGIN
  SET eligibility limits
  FOR each applicant
    CALCULATE average subject marks
    CHECK all eligibility conditions
    IF eligible, add applicant to eligible list
```

```
  SORT eligible list by marks ↓, attendance ↓, income ↑
  DISPLAY results and rank
END
```

Python Source Code

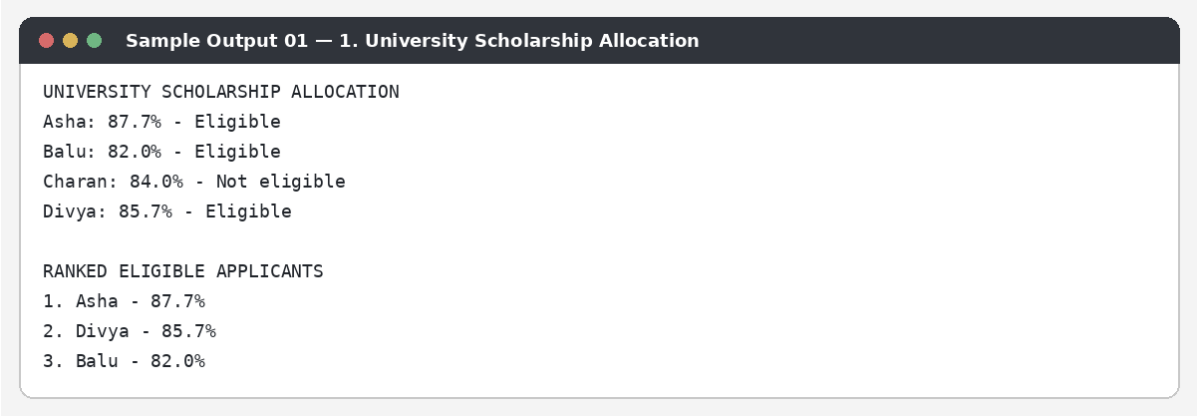
```
applicants = [
    {"name": "Asha", "marks": [88, 84, 91], "attendance": 92, "income": 180000},
    {"name": "Balu", "marks": [82, 79, 85], "attendance": 88, "income": 220000},
    {"name": "Charan", "marks": [90, 68, 94], "attendance": 95, "income": 150000},
    {"name": "Divya", "marks": [86, 87, 84], "attendance": 90, "income": 240000},
]

MIN_AGGREGATE = 80
MIN_SUBJECT = 70
MIN_ATTENDANCE = 85
MAX_INCOME = 250000

eligible = []
print("UNIVERSITY SCHOLARSHIP ALLOCATION")
for student in applicants:
    aggregate = sum(student["marks"]) / len(student["marks"])
    passed_subjects = all(m >= MIN_SUBJECT for m in student["marks"])
    ok = (aggregate >= MIN_AGGREGATE and passed_subjects
          and student["attendance"] >= MIN_ATTENDANCE
          and student["income"] <= MAX_INCOME)
    print(f'{student["name"]}: {aggregate:.1f}% - '
          f'{"Eligible" if ok else "Not eligible"}')
    if ok:
        eligible.append((student, aggregate))

eligible.sort(key=lambda item: (-item[1],
                                -item[0]["attendance"],
                                item[0]["income"]))
print("\nRANKED ELIGIBLE APPLICANTS")
for rank, (student, aggregate) in enumerate(eligible, start=1):
    print(f'{rank}. {student["name"]} - {aggregate:.1f}%')
```

Sample Output Screenshot



Test Cases

Test input / condition	Expected result
Asha	Eligible (87.7%, all rules pass)
Charan	Not eligible (one subject below 70%)
Income above ■2,50,000	Not eligible

Conclusion

The program applies scholarship conditions consistently and ranks eligible applicants using the stated tie-breaking rules.

2. Progressive Electricity Billing

Problem Statement

Calculate a household electricity bill using progressive consumption slabs, fixed charges, surcharge, and rebate. Compare monthly consumption and flag increases greater than 20%.

Objectives

• Calculate charges slab by slab.
• Add fixed charge, surcharge and rebate.
• Compare current and previous bills and flag unusual increases.

Assumptions / Rules

• Slabs: first 100 units at ■5.50; next 200 at ■7.20; next 200 at ■9.50; above 500 at ■11.00.
• Fixed charge ■150; surcharge 5% of energy charge plus fixed charge; rebate 2% of energy charge.
• An increase above 20% in consumption is flagged. Example inputs are built in.

Algorithm

1. Start.
2. Read current and previous monthly consumption.
3. Calculate energy charge using progressive slabs.
4. Calculate surcharge and rebate.
5. Calculate final bill.
6. Compare current consumption with previous consumption.
7. Flag an increase above 20%; display bill details.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Read current and previous units
↓
Calculate progressive slab charges
↓
Add fixed charge
↓
Calculate surcharge and rebate
↓
Calculate final bill
↓
Check monthly increase
↓
Display bill and warning
↓
Stop

Pseudocode

```
BEGIN
```

```
  CALCULATE charge for units in each slab
  surcharge = (energy charge + fixed charge) × 5%
  rebate = energy charge × 2%
  total = energy charge + fixed charge + surcharge − rebate
  IF consumption increase > 20%, show warning
  DISPLAY bill details
END
```

Python Source Code

```
def energy_charge(units):
    remaining = units
    charge = 0
    for slab_units, rate in [(100, 5.50), (200, 7.20),
                             (200, 9.50)]:
        used = min(remaining, slab_units)
        charge += used * rate
        remaining -= used
        if remaining <= 0:
            break
    if remaining > 0:
        charge += remaining * 11.00
    return charge

current_units = 420
previous_units = 330
energy = energy_charge(current_units)
fixed = 150
surcharge = (energy + fixed) * 0.05
rebate = energy * 0.02
total = energy + fixed + surcharge - rebate
increase = ((current_units - previous_units) / previous_units * 100
            if previous_units > 0 else 0)

print("PROGRESSIVE ELECTRICITY BILL")
print(f"Current units: {current_units} kWh")
print(f"Previous units: {previous_units} kWh")
print(f"Energy charge: Rs. {energy:.2f}")
print(f"Fixed charge: Rs. {fixed:.2f}")
print(f"Surcharge: Rs. {surcharge:.2f}")
print(f"Rebate: Rs. {rebate:.2f}")
print(f"Total bill: Rs. {total:.2f}")
print(f"Consumption increase: {increase:.1f}%")
if increase > 20:
    print("Warning: Unusual consumption increase!")
else:
    print("Consumption increase is within limit.")
```

Sample Output Screenshot

Sample Output 02 — 2. Progressive Electricity Billing

```
PROGRESSIVE ELECTRICITY BILL
Current units: 420 kWh
Previous units: 330 kWh
Energy charge: Rs. 3130.00
Fixed charge: Rs. 150.00
Surcharge: Rs. 164.00
Rebate: Rs. 62.60
Total bill: Rs. 3381.40
Consumption increase: 27.3%
Warning: Unusual consumption increase!
```

Test Cases

Test input / condition	Expected result
100 units	Energy charge = ■550.00
420 units	Energy charge = ■2,810.00
420 vs 330 units	Increase 27.3%; warning displayed

Conclusion

The program computes progressive electricity charges and highlights unusually large month-to-month consumption increases.

3. Hospital Billing and Patient Classification

Problem Statement

Calculate consultation, diagnostic, and treatment charges, apply eligible concessions, validate billing constraints, and classify patients under a simple payment policy.

Objectives

• Calculate itemized hospital charges.
• Apply concession only when eligible.
• Validate quantities and classify the payment category.

Assumptions / Rules

• Sample patient is eligible for a 10% concession on service charges.
• Emergency patients receive priority classification; otherwise bills of ■10,000 or more are classified as high-cost, and other bills as standard.
• This is an educational example, not a real hospital billing policy.

Algorithm

1. Start.
2. Read or set consultation, diagnostic and treatment charges.
3. Validate that charges are non-negative.
4. Add the charges and apply the eligible concession.
5. Classify the patient according to the stated policy.
6. Display the bill, concession, total and classification.
7. Stop.

Flowchart (vertical sequence)

Start
↓
Enter consultation, diagnostic and treatment charges
↓
Validate charges
↓
Calculate subtotal
↓
Apply eligible concession
↓
Calculate final bill
↓
Classify patient
↓
Display itemized bill
↓
Stop

Pseudocode

```
BEGIN
```

```
  VALIDATE charges ≥ 0
  subtotal = consultation + diagnostics + treatment
  concession = subtotal × concession rate if eligible
  total = subtotal − concession
  IF emergency, class = Emergency priority
  ELSE IF total ≥ 10000, class = High-cost
  ELSE class = Standard
  DISPLAY itemized bill
END
```

Python Source Code

```
consultation = 800
diagnostics = 2400
treatment = 5200
concession_rate = 0.10
concession_eligible = True
emergency = False

charges = [consultation, diagnostics, treatment]
if any(value < 0 for value in charges):
    print("Error: Charges cannot be negative.")
else:
    subtotal = sum(charges)
    concession = subtotal * concession_rate if concession_eligible else 0
    total = subtotal - concession

    if emergency:
        category = "Emergency priority"
    elif total >= 10000:
        category = "High-cost"
    else:
        category = "Standard"

    print("HOSPITAL BILL")
    print(f"Consultation: Rs. {consultation:.2f}")
    print(f"Diagnostics: Rs. {diagnostics:.2f}")
    print(f"Treatment: Rs. {treatment:.2f}")
    print(f"Subtotal: Rs. {subtotal:.2f}")
    print(f"Concession: Rs. {concession:.2f}")
    print(f"Final bill: Rs. {total:.2f}")
    print(f"Patient classification: {category}")
```

Sample Output Screenshot



Test Cases

Test input / condition	Expected result
All charges non-negative	Bill is calculated
Any negative charge	Error message; bill not calculated
Emergency patient	Emergency priority classification

Conclusion

The program produces a transparent itemized bill, applies a concession, and classifies the patient using the sample policy.

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions; and generate a formatted salary statement.

Objectives

• Calculate total earnings and gross salary.
• Apply a sample progressive deduction policy.
• Display net salary in a readable statement.

Assumptions / Rules

• HRA = 20% and other allowance = 10% of basic pay; overtime is paid at ■200 per hour.
• Incentive is supplied separately.
• Illustrative deduction: 0% on first ■15,000, 5% on the next ■15,000, and 10% on any amount above ■30,000. This is not tax advice.

Algorithm

1. Start.
2. Set basic pay, overtime hours, and incentive.
3. Calculate allowances and overtime pay.
4. Calculate gross salary.
5. Apply progressive deductions to gross salary.
6. Calculate net salary and display the statement.
7. Stop.

Flowchart (vertical sequence)

Start
↓
Read basic pay, overtime hours and incentive
↓
Calculate allowances and overtime pay
↓
Calculate gross salary
↓
Calculate progressive deduction
↓
Calculate net salary
↓
Display salary statement
↓
Stop

Pseudocode

```
BEGIN
  HRA = basic × 20%; allowance = basic × 10%
  overtime pay = hours × ■200
  gross = basic + HRA + allowance + overtime pay + incentive
  CALCULATE progressive deduction
  net = gross − deduction
```

```
  DISPLAY salary statement
END
```

Python Source Code

```
basic = 25000
overtime_hours = 6
incentive = 2000
hra = basic * 0.20
other_allowance = basic * 0.10
overtime_pay = overtime_hours * 200
gross = basic + hra + other_allowance + overtime_pay + incentive

def deduction(amount):
    first = min(amount, 15000) * 0.00
    second = min(max(amount - 15000, 0), 15000) * 0.05
    third = max(amount - 30000, 0) * 0.10
    return first + second + third

deductions = deduction(gross)
net = gross - deductions

print("EMPLOYEE SALARY STATEMENT")
print(f"Basic pay: Rs. {basic:.2f}")
print(f"HRA: Rs. {hra:.2f}")
print(f"Other allowance: Rs. {other_allowance:.2f}")
print(f"Overtime pay: Rs. {overtime_pay:.2f}")
print(f"Incentive: Rs. {incentive:.2f}")
print(f"Gross salary: Rs. {gross:.2f}")
print(f"Deductions: Rs. {deductions:.2f}")
print(f"Net salary: Rs. {net:.2f}")
```

Sample Output Screenshot

Sample Output 04 — 4. Employee Payroll and Deduction Analysis

```
EMPLOYEE SALARY STATEMENT
Basic pay: Rs. 25000.00
HRA: Rs. 5000.00
Other allowance: Rs. 2500.00
Overtime pay: Rs. 1200.00
Incentive: Rs. 2000.00
Gross salary: Rs. 35700.00
Deductions: Rs. 1320.00
Net salary: Rs. 34380.00
```

Test Cases

Test input / condition	Expected result
Basic ■25,000; 6 overtime hours	Gross = ■35,700
Gross ■15,000	Deduction = ■0
Gross ■35,700	Deduction = ■820

Conclusion

The program calculates gross earnings, applies progressive deductions, and presents the employee’s net salary.

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans against a user's usage, validity, and budget; calculate excess usage charges; and identify the lowest-cost plan that meets the requirements.

Objectives

• Estimate the total cost of each plan.
• Reject plans that fail validity or budget requirements.
• Select the cheapest eligible plan.

Assumptions / Rules

• Usage is measured in GB and validity in days.
• Excess data is charged at the plan's listed per-GB rate.
• Sample rates are illustrative, not current telecom offers.

Algorithm

1. Start.
2. Set required data, validity and budget.
3. Store plan details and calculate any excess-data charge.
4. Add plan price and excess charge.
5. Keep plans meeting validity and budget requirements.
6. Choose the eligible plan with the lowest total cost.
7. Display comparison and best plan.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Enter data use, validity and budget
↓
Read mobile plan details
↓
Calculate excess data charges
↓
Calculate total plan cost
↓
Check validity and budget
↓
Choose lowest-cost eligible plan
↓
Display comparison
↓
Stop

Pseudocode

```
BEGIN
```

```
  FOR each plan
    excess = max(required data − included data, 0)
    total = price + excess × extra-data rate
    IF validity sufficient AND total ≤ budget, keep plan
  SELECT kept plan with minimum total cost
  DISPLAY results
END
```

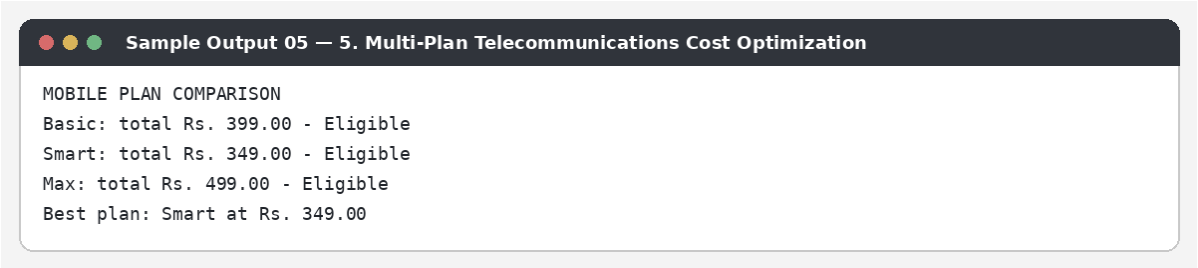
Python Source Code

```
required_data = 18
required_days = 28
budget = 500
plans = [
    {"name": "Basic", "price": 199, "data": 10, "days": 28, "extra": 25},
    {"name": "Smart", "price": 349, "data": 20, "days": 28, "extra": 20},
    {"name": "Max", "price": 499, "data": 30, "days": 56, "extra": 15},
]

eligible = []
print("MOBILE PLAN COMPARISON")
for plan in plans:
    excess = max(required_data - plan["data"], 0)
    total = plan["price"] + excess * plan["extra"]
    meets = plan["days"] >= required_days and total <= budget
    print(f'{plan["name"]}: total Rs. {total:.2f} - '
          f'{"Eligible" if meets else "Not eligible"}')
    if meets:
        eligible.append((total, plan["name"]))

if eligible:
    cost, name = min(eligible)
    print(f"Best plan: {name} at Rs. {cost:.2f}")
else:
    print("No plan meets all requirements.")
```

Sample Output Screenshot



Test Cases

Test input / condition	Expected result
18 GB, 28 days, ■500 budget	Smart plan selected at ■349
Budget ■300	No plan eligible because even the cheapest qualifying total exceeds budget
Required validity 60 days	No plan eligible because no plan meets validity and budget requirements

Conclusion

The program compares total estimated costs while respecting the user’s data, validity, and budget requirements.

6. Railway Fare and Reservations Calculator

Problem Statement

Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and cancellation penalties.

Objectives

• Calculate distance-based base fare.
• Apply class multipliers and age concessions.
• Add reservation fees and estimate cancellation penalties.

Assumptions / Rules

• Base fare: ■1.50/km for first 100 km, ■1.20/km for next 400 km, and ■1.00/km above 500 km.
• Class multipliers: Sleeper 1.0, AC 2.0; reservation fee ■40 per passenger.
• Children aged 5–11 receive 50% fare concession; passengers aged 60 or above receive 30% concession. Cancellation penalty is 25% of fare before reservation fee. Illustrative rules only.

Algorithm

1. Start.
2. Set journey distance, class, and passenger ages.
3. Calculate distance-slab base fare.
4. Apply class multiplier and age concession for each passenger.
5. Add reservation fee per passenger.
6. Calculate total booking cost and sample cancellation penalty.
7. Display fare details.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Enter distance, class and passenger ages
↓
Calculate distance-slab fare
↓
Apply class multiplier
↓
Apply age concessions
↓
Add reservation fees
↓
Calculate booking and cancellation estimate
↓
Display fare details
↓
Stop

Pseudocode

```
BEGIN
  CALCULATE distance slab fare
  APPLY class multiplier
  FOR each passenger
    APPLY age concession if eligible
    passenger total = adjusted fare + reservation fee
  total booking = sum passenger totals
  cancellation estimate = sum(adjusted fare × 25%)
  DISPLAY booking details
END
```

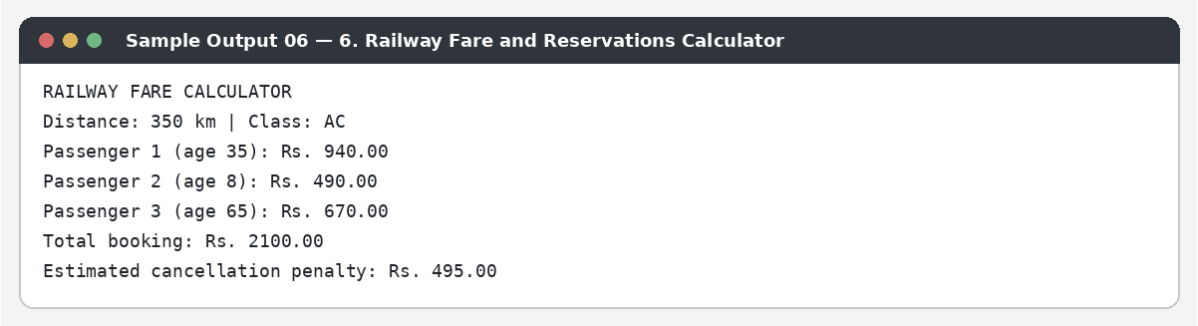
Python Source Code

```
distance_km = 350
travel_class = "AC"
passenger_ages = [35, 8, 65]
reservation_fee = 40
class_multiplier = {"Sleeper": 1.0, "AC": 2.0}

def base_fare(distance):
    first = min(distance, 100) * 1.50
    second = min(max(distance - 100, 0), 400) * 1.20
    third = max(distance - 500, 0) * 1.00
    return first + second + third

base = base_fare(distance_km) * class_multiplier[travel_class]
booking_total = 0
cancellation_total = 0
print("RAILWAY FARE CALCULATOR")
print(f"Distance: {distance_km} km | Class: {travel_class}")
for index, age in enumerate(passenger_ages, start=1):
    if age < 5:
        concession = 1.00
    elif 5 <= age <= 11:
        concession = 0.50
    elif age >= 60:
        concession = 0.30
    else:
        concession = 0.00
    fare = base * (1 - concession)
    total = fare + reservation_fee
    booking_total += total
    cancellation_total += fare * 0.25
    print(f"Passenger {index} (age {age}): Rs. {total:.2f}")
print(f"Total booking: Rs. {booking_total:.2f}")
print(f"Estimated cancellation penalty: Rs. {cancellation_total:.2f}")
```

Sample Output Screenshot



Test Cases

Test input / condition	Expected result
350 km, AC, age 35	Fare including reservation = ■760
Passenger age 8	50% concession applied
Passenger age 65	30% concession applied

Conclusion

The program estimates fares for several passengers and demonstrates how class, distance, concessions, and reservation fees affect the total.

7. Retail Billing with Multiple Discount Rules

Problem Statement

Process multiple products and quantities, apply category-specific taxes and threshold discounts, enforce a maximum concession limit, and generate an itemized invoice.

Objectives

• Calculate each item's subtotal and tax.
• Apply a bill-level discount only when the threshold is met.
• Limit the discount and print an itemized invoice.

Assumptions / Rules

• Sample tax rates: food 5%, clothing 12%, electronics 18%.
• A 10% discount applies when pre-tax subtotal is at least ■2,000, capped at ■500.
• For simplicity, discount is subtracted after tax; a real store should follow its applicable tax rules.

Algorithm

1. Start.
2. Store product name, category, unit price and quantity.
3. Validate that price and quantity are non-negative.
4. Calculate each line subtotal and category tax.
5. Add line totals and calculate the eligible discount.
6. Apply the maximum discount cap and calculate final total.
7. Display itemized invoice.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Enter product and quantity details
↓
Validate prices and quantities
↓
Calculate line subtotals
↓
Calculate category-specific taxes
↓
Apply threshold discount and cap
↓
Calculate final invoice
↓
Display itemized bill
↓
Stop

Pseudocode

```
BEGIN
```

```
  subtotal = 0; tax_total = 0
  FOR each product
    line = price × quantity
    tax = line × category tax rate
    add line and tax to totals
  IF subtotal ≥ ■2000, discount = min(subtotal × 10%, ■500)
  final = subtotal + tax_total − discount
  DISPLAY itemized invoice
END
```

Python Source Code

```
products = [
    {"name": "Rice Pack", "category": "food", "price": 250, "qty": 2},
    {"name": "Shirt", "category": "clothing", "price": 800, "qty": 2},
    {"name": "Headphones", "category": "electronics", "price": 1200, "qty": 1},
]
tax_rates = {"food": 0.05, "clothing": 0.12, "electronics": 0.18}
subtotal = 0
tax_total = 0

print("RETAIL INVOICE")
for item in products:
    if item["price"] < 0 or item["qty"] < 0:
        raise ValueError("Price and quantity cannot be negative.")
    line = item["price"] * item["qty"]
    tax = line * tax_rates[item["category"]]
    subtotal += line
    tax_total += tax
    print(f'{item["name"]}: {item["qty"]} × Rs. {item["price"]:.2f} '
          f'= Rs. {line:.2f}; tax Rs. {tax:.2f}')

discount = min(subtotal * 0.10, 500) if subtotal >= 2000 else 0
final_total = subtotal + tax_total - discount
print(f"Subtotal: Rs. {subtotal:.2f}")
print(f"Tax total: Rs. {tax_total:.2f}")
print(f"Discount: Rs. {discount:.2f}")
print(f"Final total: Rs. {final_total:.2f}")
```

Sample Output Screenshot

Sample Output 07 — 7. Retail Billing with Multiple Discount Rules

```
RETAIL INVOICE
Rice Pack: 2 × Rs. 250.00 = Rs. 500.00; tax Rs. 25.00
Shirt: 2 × Rs. 800.00 = Rs. 1600.00; tax Rs. 192.00
Headphones: 1 × Rs. 1200.00 = Rs. 1200.00; tax Rs. 216.00
Subtotal: Rs. 3300.00
Tax total: Rs. 433.00
Discount: Rs. 330.00
Final total: Rs. 3403.00
```

Test Cases

Test input / condition	Expected result
Subtotal ■3,300	10% discount = ■330
Subtotal below ■2,000	No discount
Negative price or quantity	Validation error

Conclusion

The program creates an itemized retail invoice with category taxes and a capped threshold discount.

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material, labour, wastage, and machine costs; determine the selling price and check whether production meets a specified profit target.

Objectives

• Calculate the complete production cost.
• Estimate selling price and profit margin.
• Check whether the target profit percentage is achieved.

Assumptions / Rules

• Wastage is calculated as a percentage of material cost.
• Profit target is measured as a percentage of total production cost.
• The example uses a batch of 100 units.

Algorithm

1. Start.
2. Set material, labour, machine cost, quantity, wastage rate, and target profit.
3. Calculate wastage cost and total production cost.
4. Calculate cost per unit.
5. Calculate target selling price and planned profit.
6. Compare actual planned profit percentage with the target.
7. Display the result.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Enter material, labour, machine, quantity and target profit
↓
Calculate wastage cost
↓
Calculate total production cost
↓
Calculate cost per unit
↓
Calculate target selling price
↓
Check profit target
↓
Display cost and profitability report
↓
Stop

Pseudocode

```
BEGIN
```

```
  wastage = material cost × wastage rate
  total cost = material + labour + machine + wastage
  unit cost = total cost ÷ quantity
  target selling price = unit cost × (1 + target profit rate)
  planned profit = selling price × quantity − total cost
  CHECK target profit and DISPLAY report
END
```

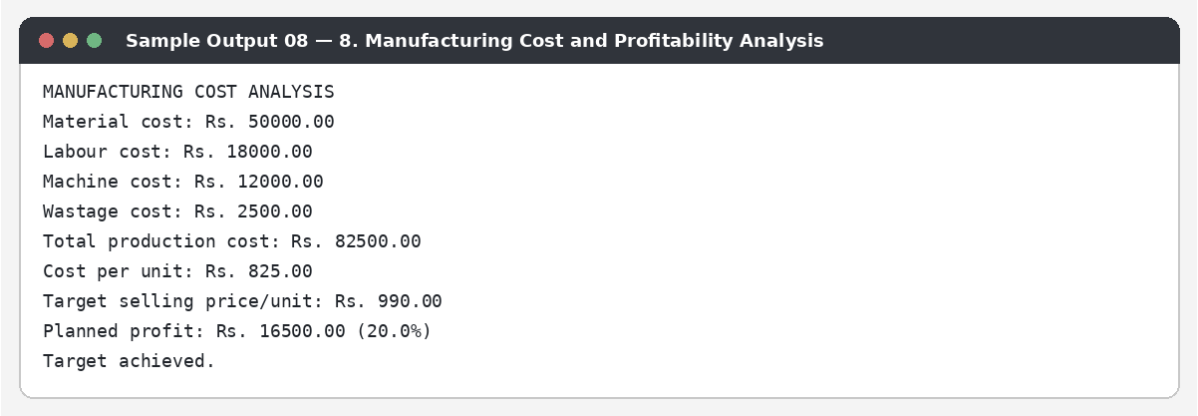
Python Source Code

```
material_cost = 50000
labour_cost = 18000
machine_cost = 12000
wastage_rate = 0.05
quantity = 100
target_profit_rate = 0.20

if quantity <= 0 or min(material_cost, labour_cost, machine_cost) < 0:
    print("Error: Quantity must be positive and costs cannot be negative.")
else:
    wastage_cost = material_cost * wastage_rate
    total_cost = material_cost + labour_cost + machine_cost + wastage_cost
    unit_cost = total_cost / quantity
    target_price_per_unit = unit_cost * (1 + target_profit_rate)
    total_sales = target_price_per_unit * quantity
    profit = total_sales - total_cost
    actual_profit_rate = profit / total_cost * 100

    print("MANUFACTURING COST ANALYSIS")
    print(f"Material cost: Rs. {material_cost:.2f}")
    print(f"Labour cost: Rs. {labour_cost:.2f}")
    print(f"Machine cost: Rs. {machine_cost:.2f}")
    print(f"Wastage cost: Rs. {wastage_cost:.2f}")
    print(f"Total production cost: Rs. {total_cost:.2f}")
    print(f"Cost per unit: Rs. {unit_cost:.2f}")
    print(f"Target selling price/unit: Rs. {target_price_per_unit:.2f}")
    print(f"Planned profit: Rs. {profit:.2f} ({actual_profit_rate:.1f}%)")
    print("Target achieved." if actual_profit_rate >= target_profit_rate * 100
          else "Target not achieved.")
```

Sample Output Screenshot



Test Cases

Test input / condition	Expected result
100 units, target 20%	Unit cost ■825; target price ■990
Quantity = 0	Error message
Negative machine cost	Error message

Conclusion

The program estimates production costs and the selling price required to meet a chosen profit target.

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert travel expenses using supplied exchange rates, include conversion fees, compare planned spending with a budget, and identify the largest expense category.

Objectives

• Convert expenses into a base currency.
• Include conversion fees in the budget total.
• Find the largest category and check the budget.

Assumptions / Rules

• All exchange rates are user-supplied examples expressed as INR per 1 unit of foreign currency.
• Conversion fee is 2% of the converted INR amount.
• Rates are illustrative and should be replaced with current supplied rates for real travel planning.

Algorithm

1. Start.
2. Set the INR budget, conversion fee, exchange rates, and expense categories.
3. Convert each expense into INR.
4. Calculate the conversion fee and total converted cost per category.
5. Add expenses and compare with the budget.
6. Find the largest expense category.
7. Display the summary.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Enter budget, exchange rates and travel expenses
↓
Convert expenses to INR
↓
Calculate conversion fees
↓
Calculate total spending
↓
Compare spending with budget
↓
Find largest expense category
↓
Display budget report
↓
Stop

Pseudocode

```
BEGIN
```

```
  FOR each category expense
    converted = foreign amount × supplied INR exchange rate
    fee = converted × fee rate
    category total = converted + fee
  total spending = sum(category totals)
  largest category = category with maximum total
  compare spending with budget and DISPLAY results
END
```

Python Source Code

```
budget_inr = 60000
fee_rate = 0.02
# Example rates: INR per 1 unit of foreign currency
rates = {"USD": 83.0, "EUR": 90.0, "GBP": 105.0}
expenses = [
    {"category": "Hotel", "amount": 250, "currency": "USD"},
    {"category": "Food", "amount": 100, "currency": "EUR"},
    {"category": "Transport", "amount": 80, "currency": "USD"},
    {"category": "Tickets", "amount": 100, "currency": "GBP"},
]

totals = {}
print("TRAVEL BUDGET ANALYSIS")
for item in expenses:
    converted = item["amount"] * rates[item["currency"]]
    fee = converted * fee_rate
    total = converted + fee
    category = item["category"]
    totals[category] = total
    print(f'{category}: {item["amount"]} {item["currency"]} '
          f'= Rs. {total:.2f} including fee')

spending = sum(totals.values())
largest = max(totals, key=totals.get)
print(f"Total spending: Rs. {spending:.2f}")
print(f"Budget: Rs. {budget_inr:.2f}")
print(f"Remaining: Rs. {budget_inr - spending:.2f}")
print(f"Largest expense: {largest} (Rs. {totals[largest]:.2f})")
print("Within budget." if spending <= budget_inr else "Budget exceeded!")
```

Sample Output Screenshot

Sample Output 09 — 9. Multi-Currency Travel Budget Analysis

```
TRAVEL BUDGET ANALYSIS
Hotel: 250 USD = Rs. 21165.00 including fee
Food: 100 EUR = Rs. 9180.00 including fee
Transport: 80 USD = Rs. 6772.80 including fee
Tickets: 100 GBP = Rs. 10710.00 including fee
Total spending: Rs. 47827.80
Budget: Rs. 60000.00
Remaining: Rs. 12172.20
Largest expense: Hotel (Rs. 21165.00)
Within budget.
```

Test Cases

Test input / condition	Expected result
250 USD at ■83 plus 2% fee	■21,165
Total spending ■47,827.80	Within ■60,000 budget
Hotel category	Largest expense

Conclusion

The program estimates foreign-currency travel costs in INR, adds conversion fees, and identifies the largest expense category.

10. Campus Event Financial Planning

Problem Statement

Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions and sponsorship shortfall; and find the minimum registrations needed to break even.

Objectives

• Calculate the full event cost.
• Subtract sponsorship and calculate the amount participants must contribute.
• Estimate the minimum registration count needed to cover costs.

Assumptions / Rules

• Fixed costs include venue, equipment, and transport. Catering and certificates are per participant.
• Participant contribution per registration is ■500.
• Sponsorship is a fixed amount. Break-even registrations are rounded up to the next whole participant.

Algorithm

1. Start.
2. Set fixed costs, per-participant costs, sponsorship, and participant contribution.
3. Calculate fixed and variable costs for the expected participants.
4. Calculate total event cost and sponsorship shortfall.
5. Calculate the amount that must be raised from registrations.
6. Calculate the break-even registration count using ceiling division.
7. Display the financial plan.
8. Stop.

Flowchart (vertical sequence)

Start
↓
Enter event costs, sponsorship and registration fee
↓
Calculate fixed and variable costs
↓
Calculate total event cost
↓
Subtract sponsorship
↓
Calculate registration shortfall
↓
Calculate minimum break-even registrations
↓
Display financial plan
↓
Stop

Pseudocode

```
BEGIN
```

```
  fixed = venue + equipment + transport
  variable per person = catering + certificate
  total cost = fixed + variable per person × expected participants
  shortfall after sponsorship = max(total cost − sponsorship, 0)
  registration revenue needed = max(total cost − sponsorship, 0)
  break-even count = ceiling((fixed − sponsorship) ÷ (fee − variable per person))
  DISPLAY plan and check whether fee covers variable cost
END
```

Python Source Code

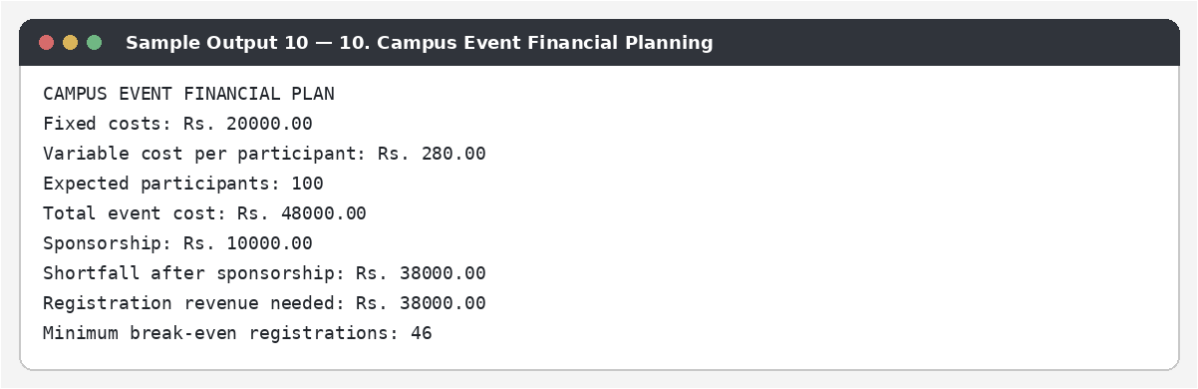
```
venue = 12000
equipment = 5000
transport = 3000
catering_per_person = 250
certificate_per_person = 30
expected_participants = 100
sponsorship = 10000
registration_fee = 500

fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + variable_per_person * expected_participants
shortfall = max(total_cost - sponsorship, 0)
registration_revenue_needed = shortfall

contribution_margin = registration_fee - variable_per_person
if contribution_margin <= 0:
    break_even = None
else:
    break_even = max(0, math.ceil((fixed_cost - sponsorship) /
                                  contribution_margin))

print("CAMPUS EVENT FINANCIAL PLAN")
print(f"Fixed costs: Rs. {fixed_cost:.2f}")
print(f"Variable cost per participant: Rs. {variable_per_person:.2f}")
print(f"Expected participants: {expected_participants}")
print(f"Total event cost: Rs. {total_cost:.2f}")
print(f"Sponsorship: Rs. {sponsorship:.2f}")
print(f"Shortfall after sponsorship: Rs. {shortfall:.2f}")
print(f"Registration revenue needed: Rs. {registration_revenue_needed:.2f}")
if break_even is None:
    print("Break-even impossible: registration fee does not cover variable cost.")
else:
    print(f"Minimum break-even registrations: {break_even}")
```

Sample Output Screenshot



Test Cases

Test input / condition	Expected result
100 participants	Total cost ■48,000
Sponsorship ■10,000	Shortfall ■38,000
Fee ≤ variable cost	Break-even impossible message

Conclusion

The program helps organizers estimate event expenses, calculate the funding gap, and plan a minimum registration target.